

Bachelorarbeit: Low-Power IoT-Skala

Thomas Rocheteau

Februar 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	Ausgangslage	3
1.2	Zielsetzung	3
1.3	Meilensteine	3
2	Technische Ausgangslage	3
2.1	Design Space Exploration	3
2.1.1	MCU-Auswahl: ESP32-S3 vs. ESP32-C3	3
2.1.2	Evaluation kommerzieller Boards	3
2.1.3	Kommunikationsstrategie	3
3	Optimierung Messgenauigkeit	3
3.1	Sensorik: Wägezellen und ADC	3
3.1.1	Wägezellen (YZC-133)	3
3.1.2	ADS1234 – 4-Kanal-24-Bit-ADC	4
3.1.3	Von einem auf vier ADC-Kanäle	4
3.2	ADS1234 – Ansteuerung und Kalibrierung	5
3.2.1	Überblick	5
3.2.2	Seriellles Leseprotokoll	5
3.2.3	Kanalwechsel CH4 → CH1: PDWN-Workaround	5
3.2.4	Kalibrierung	6
3.3	Vergleich Messgenauigkeit	7
3.3.1	Vorgängerarbeit Messreihe	7
3.3.2	Optimiertes Design Messreihe	7
4	Low-Power Optimierung	7
4.1	Motivation	7
4.2	Strompfad Schema	7
4.2.1	Ausgangslage	8
4.2.2	Hardware Redesign	8
4.3	Strommessungen und Low-Power-Optimierung	9
4.4	Übersicht Zyklus	9
4.5	Software Optimierung	10
4.6	Gesamtsystem Stromverbrauch	11
4.6.1	Vergleich Energieverbrauch Aktiver Zyklus	11
4.6.2	Vergleich Leistungsverbrauch Deep-Sleep	12

4.6.3	Vergleich Energieverbrauch ESP32	12
4.7	Verluste Hardware	15
4.7.1	Methodik	16
4.7.2	Buck converter	16
4.7.3	ADC	17
4.8	Gesamtverbrauch und Batterielebensdauer	17

1 Einleitung

1.1 Ausgangslage

1.2 Zielsetzung

1.3 Meilensteine

1. **Hardware-Entwicklung:** Custom ESP32 Low-Power Board vs. kommerzielle Lösungen.
2. **Quality of Life Improvements:** Implementierung von Bedienelementen (Taster) sowie Software-Funktionen (Tara, Kalibrierung).
3. **Kommunikationsprotokolle:** Evaluation von ESP-NOW vs. BLE.

2 Technische Ausgangslage

2.1 Design Space Exploration

2.1.1 MCU-Auswahl: ESP32-S3 vs. ESP32-C3

- **ESP32-S3:** Verfügt über einen Ultra-Low-Power (ULP) Co-Prozessor. Ermöglicht präzises Abtasten und Detektion von Gewichtsänderungen im Tiefschlaf. Datasheet sagt $8\mu A$ [3][2].
- **ESP32-C3:** Intervall-basiertes Abtasten (z. B. alle 5 Minuten) durch periodisches Aufwachen des Hauptprozessors. Datasheet sagt $5\mu A$ [1] - ESP32-C3 integrates two 12-bit SAR ADCs.

2.1.2 Evaluation kommerzieller Boards

- **ESP32-C3 Super Mini:** Spezifiziert mit einem Low-Power-Stromverbrauch von ca. $43\mu A$ (im Deep Sleep).

2.1.3 Kommunikationsstrategie

Untersuchung der Vor- und Nachteile von **ESP-NOW** gegenüber **BLE** (Bluetooth Low Energy).

3 Optimierung Messgenauigkeit

3.1 Sensorik: Wägezellen und ADC

3.1.1 Wägezellen (YZC-133)

Als Kraftsensoren kommen vier Wägezellen des Typs YZC-133 zum Einsatz [4]. Jede Zelle ist für eine Nennlast von 1 kg ausgelegt und enthält intern eine vollständige Wheatstone-Brücke aus vier Dehnungsmessstreifen (DMS). Die relevanten elektrischen Kenngrößen sind in Tabelle 1 zusammengefasst.

Bei 3.3 V Erregerspannung liegen beide Ausgangspins der Brücke auf einem Gleichaktpotenzial von ca. 1.65 V ($V_{\text{ex}}/2$). Das Nutzsignal ist die *Differenz* zwischen diesen

Tabelle 1: Elektrische Spezifikation YZC-133

Parameter	Wert
Nennlast	1 kg
Nennkennwert (Empfindlichkeit)	$1,0 \pm 0,15 \text{ mV/V}$
Nullpunktsignal	$\pm 0,1 \text{ mV/V}$
Kriechfehler	$0,03 \% \text{ F.S./30 min}$
Eingangsimpedanz	$1066 \pm 20 \Omega$
Ausgangsimpedanz	$1000 \pm 5 \Omega$

beiden Pins: bei Vollast nominell $1,0 \text{ mV/V} \times 3,3 \text{ V} = 3.3 \text{ mV}$. Der differentielle Eingang des ADS1234 erfasst genau diese Differenz und unterdrückt das Gleichtaktsignal durch seine hohe Gleichtaktunterdrückung (CMRR). Die vier Zellen sind in X-Formation an den Ecken der Waagplattform montiert.

3.1.2 ADS1234 – 4-Kanal-24-Bit-ADC

Für die Signalerfassung wird der ADS1234 von Texas Instruments eingesetzt [6]. Der Chip ist ein 24-Bit-Sigma-Delta-ADC mit vier differentiellen Eingangskanälen und einem integrierten programmierbaren Verstärker (PGA) mit den Verstärkungsfaktoren 1, 2, 64 und 128. Mit PGA-Gain 128 ergibt sich für das 3.3 mV -Signal der YZC-133 ein verstärktes Differenzsignal von ca. 422 mV , das den Eingangsbereich des ADC gut ausnutzt. Die wählbare Ausgabedatenrate beträgt 10 SPS oder 80 SPS; im vorliegenden System wird 10 SPS verwendet, da dies das Eingangsrauschen durch die längere Integrationszeit maximal reduziert.

3.1.3 Von einem auf vier ADC-Kanäle

In der Vorarbeit zu dieser Bachelorarbeit waren alle vier Wägezellen in einer gemeinsamen Wheatstone-Brücke zusammengeschaltet und lieferten ihr summiertes Signal an einen einzigen ADC-Kanal. In dieser Arbeit wird jede Wägezelle einzeln an einen eigenen Eingangskanal des ADS1234 geführt. Dieser Wechsel bringt zwei wesentliche Vorteile:

Ecklastkompensation Mit einem einzigen Summensignal ist keine positionsabhängige Korrektur möglich: Das System kann nicht unterscheiden, ob eine Abweichung durch einen exzentrischen Lastangriffspunkt oder durch eine tatsächliche Massenänderung entsteht. Durch die individuellen Kanäle lässt sich jede Wägezelle separat kalibrieren und der dominante Sensor pro Lastposition gezielt korrigieren (siehe Abschnitt 3.2).

Rauschverhalten und individuelle Kalibrierung Die Empfindlichkeit der YZC-133 streut mit $\pm 15 \%$ zwischen den Exemplaren. In der Einzelkanal-Lösung mittelte sich diese Streuung über die parallele Zusammenschaltung heraus, konnte aber nicht pro Sensor korrigiert werden. Mit vier separaten Kanälen wird die Empfindlichkeitsdifferenz jeder Wägezelle durch den individuell bestimmten Skalierungsfaktor (`spanFactor`) in der Software exakt kompensiert. Zusätzlich verbessert die digitale Summation vier statistisch unkorrelierter ADC-Messungen das Signal-Rausch-Verhältnis gegenüber einer einzigen Messung um den Faktor $\sqrt{4} = 2$ (+6 dB), da sich das Rauschen der Kanäle inkohärent addiert, während sich die Nutzsignale kohärent addieren.

3.2 ADS1234 – Ansteuerung und Kalibrierung

3.2.1 Überblick

Der ADS1234 ist ein 24-Bit-Sigma-Delta-ADC von Texas Instruments, der speziell für den Einsatz mit Brückensensoren wie Dehnungsmessstreifen (DMS) ausgelegt ist [6]. Er stellt vier unabhängige Eingangskanäle bereit, die über zwei digitale Adressleitungen (A0, A1) ausgewählt werden. Die Kommunikation mit dem Mikrocontroller erfolgt über ein synchrones serielles Protokoll mit den Leitungen DRDY/DOUT (kombiniertes Datenbereit- und Datensignal), SCLK (Takt) sowie PDWN (Power-Down/Reset).

Tabelle 2: Pin-Belegung ADS1234 – ESP32

Signal	GPIO	Funktion
DRDY/DOUT	2	Datenbereit-Anzeige und serieller Datenausgang
SCLK	15	Serieller Takt (Master-Ausgang)
PDWN	26	Power-Down / Reset
DMS_PWR	17	Versorgung der Wägezellen (schaltbar)
A0	32	Kanal-Adresse Bit 0
A1	4	Kanal-Adresse Bit 1

3.2.2 Serielles Leseprotokoll

Der ADS1234 signalisiert eine abgeschlossene Wandlung, indem er die DRDY-Leitung auf LOW zieht. Die Software wartet aktiv auf diesen Zustand und liest anschliessend 24 Bit MSB-first aus, indem für jedes Bit ein SCLK-Impuls erzeugt und der Pegel von DOUT abgetastet wird. Nach dem 24. Bit wird gemäss Datenblatt (Figure 7-10) ein 25. Taktimpuls gesendet, der DRDY wieder auf HIGH zwingt und damit den ADC für die nächste Wandlung freigibt. Da der ADS1234 ein Zweierkomplement-Format mit 24 Bit Breite verwendet, wird der Rohwert abschliessend vorzeichenrichtig auf 32 Bit erweitert:

```
if (raw & 0x800000) raw |= 0xFF000000;
return (int32_t)raw;
```

3.2.3 Kanalwechsel CH4 → CH1: PDWN-Workaround

Der ADS1234 kodiert den aktiven Kanal über die beiden Adressleitungen als 2-Bit-Binärzahl: Kanal 1 entspricht A0=0, A1=0, Kanal 4 entspricht A0=1, A1=1. Der Chip erkennt einen Kanalwechsel intern anhand steigender Flanken auf A0 oder A1. Beim Wechsel von Kanal 4 nach Kanal 1 fallen jedoch *beide* Adressleitungen von HIGH auf LOW – es entsteht keine einzige steigende Flanke. Der ADS1234 erkennt diesen Wechsel daher nicht als Kanalumschaltung und setzt DRDY nicht zurück. Die Leitung bleibt LOW, was der Software fälschlicherweise signalisiert, dass bereits ein gültiger Messwert des neuen Kanals vorliegt, obwohl noch der alte Kanal aktiv ist.

Als Lösung wird nach dem Setzen der Adressleitungen auf Kanal 1 geprüft, ob DRDY bereits LOW ist. Trifft dies zu, wird ein PDWN-Puls von 500 µs ausgelöst, der den ADC in den Power-Down-Zustand versetzt und anschliessend neu startet. Dieser Reset zwingt

den Chip, den ausgewählten Kanal neu zu initialisieren und eine frische Wandlung durchzuführen, bevor DRDY das nächste Mal LOW wird.

```
int32_t readChannel(int ch) {
    setChannel(ch);
    if (ch == 1 && digitalRead(PIN_DRDY_DOUT) == LOW) {
        digitalWrite(PIN_PDWN, LOW);
        delayMicroseconds(500);
        digitalWrite(PIN_PDWN, HIGH);
    }
    return readADS1234();
}
```

3.2.4 Kalibrierung

Die Kalibrierung läuft in drei aufeinanderfolgenden Schritten ab und wird einmalig beim Start der Firmware über die serielle Schnittstelle ausgelöst. Alle Messwerte sind Mittelwerte aus `CAL_SAMPLES = 5` Wandlungen pro Kanal, was bei 10 SPS einer Messzeit von rund 0,4 Sekunden pro Kanal entspricht.

Schritt 1 – Nullpunkt (Zero Offset) Die Waage wird vollständig entlastet und für jeden der vier Kanäle ein gemittelter Rohwert erfasst, der als Nullpunkt `zeroOffset[ch]` gespeichert wird. Dieser Wert kompensiert thermische und mechanische Vorlast der Wägezellen sowie Offset-Fehler des ADC.

Schritt 2 – Empfindlichkeit (Span) Ein bekanntes Kalibriergewicht von 3000 g wird mittig auf die Waagplattform gelegt. Für jeden Kanal wird die Abweichung vom Nullpunkt berechnet:

$$\Delta_i = \bar{x}_i - \text{zeroOffset}_i \quad (1)$$

Die vier Wägezellen sind in X-Formation an den Ecken der Plattform angebracht. Bei dieser Anordnung werden die zwei diagonal gegenüberliegenden Zellen beim Belasten auf Zug, die anderen zwei auf Druck beansprucht, was zu einem mechanisch bedingten Vorzeichenwechsel im ADC-Signal führt. Das Vorzeichen von Δ_i wird daher pro Kanal automatisch erfasst und als `channelSign` gespeichert. Der initiale Skalierungsfaktor ergibt sich aus der Gesamtauslenkung über alle vier Kanäle:

$$s_{\text{init}} = \frac{m_{\text{cal}}}{\sum_{i=1}^4 |\Delta_i|} \quad \Rightarrow \quad \text{spanFactor}_i = \text{channelSign}_i \cdot s_{\text{init}} \quad (2)$$

Damit trägt jeder Kanal anteilig zur Gesamtmasse bei, wobei die Polarität korrekt berücksichtigt wird.

Schritt 3 – Ecklastkompensation (Corner Trim) Bei einer Plattformwaage mit vier Wägezellen in den Ecken führen fertigungsbedingte Toleranzen der Sensoren dazu, dass ein exzentrisch aufgelegtes Gewicht – je nach Position – unterschiedliche Gesamtmassen anzeigt. Um dies zu kompensieren, wird das Kalibriergewicht nacheinander über jede der vier Ecken gestellt.

Bei jeder Eckenbelastung ermittelt die Software denjenigen Kanal mit der grössten absoluten Auslenkung (*dominanter Kanal*), da dieser am stärksten von der Lastposition beeinflusst wird. Die aktuelle Gesamtanzeige m_{meas} ergibt sich aus den Beiträgen aller vier Kanäle:

$$m_{\text{meas}} = \sum_{i=1}^4 \text{spanFactor}_i \cdot \Delta_i \quad (3)$$

Der Skalierungsfaktor des dominanten Kanals d wird anschliessend so angepasst, dass die Anzeige exakt dem Kalibriergewicht entspricht, während die Faktoren der übrigen Kanäle unverändert bleiben:

$$\text{spanFactor}_d = \frac{m_{\text{cal}} - \sum_{i \neq d} \text{spanFactor}_i \cdot \Delta_i}{\Delta_d} \quad (4)$$

Ein einziger Durchlauf über alle vier Ecken ist für typische DMS-Toleranzen von unter 5 % ausreichend.

Gewichtsberechnung im Betrieb Nach abgeschlossener Kalibrierung berechnet sich die angezeigte Masse in jedem Messtakt als:

$$m = \sum_{i=1}^4 \text{spanFactor}_i \cdot (\text{raw}_i - \text{zeroOffset}_i) \quad (5)$$

3.3 Vergleich Messgenauigkeit

3.3.1 Vorgängerarbeit Messreihe

3.3.2 Optimierte Design Messreihe

4 Low-Power Optimierung

4.1 Motivation

Die Waage ist für den Einsatz in der Intralogistik konzipiert, wo sie autonom und ohne regelmässige Wartung den Füllstand von Lagerbehältern überwacht. In diesem Umfeld ist eine möglichst lange Batterielaufzeit entscheidend: Jeder Akkuwechsel verursacht Wartungsaufwand, der bei einer grossen Anzahl verteilter Geräte erhebliche Betriebskosten erzeugen kann.

Da die Vorgängerarbeit ihren Schwerpunkt auf die funktionale Korrektheit des Systems legte, wurde die Optimierung des Stromverbrauchs bisher nicht adressiert. Das System bietet daher noch erhebliches Einsparpotenzial, das in dieser Arbeit systematisch erschlossen wird.

4.2 Strompfad Schema

Die folgenden Diagramme zeigen den Strompfad der alten und neuen Waage. Das Power Profiler Kit II (PPK2) als Messinstrument wird im nachfolgenden Unterkapitel vorgestellt.

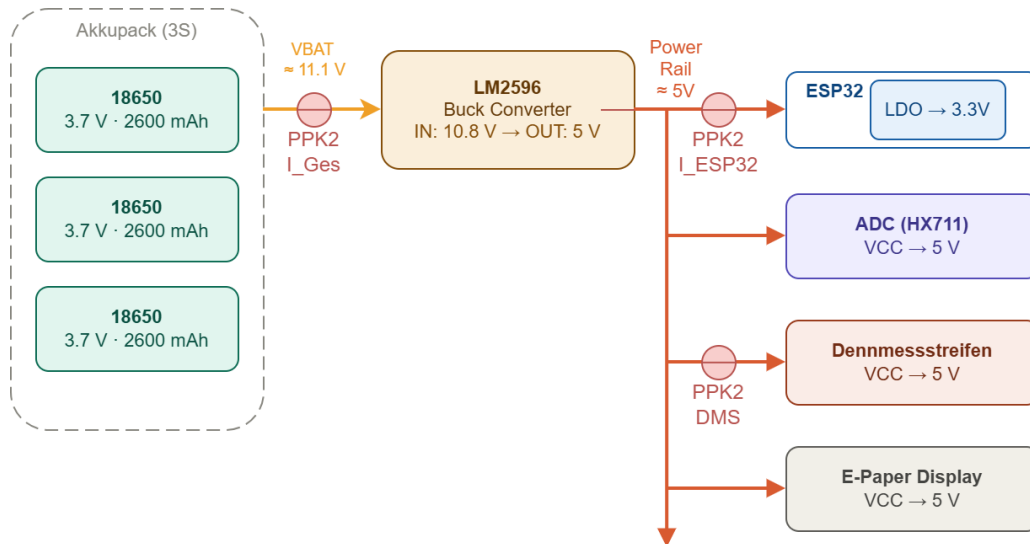


Abbildung 1: Schema Strompfad – Ausgangslage

4.2.1 Ausgangslage

Die Spannungsversorgung der Ausgangslage besteht aus drei in Reihe geschalteten 18650-Lithium-Ionen-Zellen mit einer Nennspannung von 11.1 V. Ein **LM2596**-Schaltregler wandelt diese Spannung auf ein stabiles 5 V-Rail herunter, von dem alle Verbraucher gespeist werden: der ESP32 (über den internen LDO des Entwicklerboards), der HX711-ADC, die Dehnmessstreifen sowie das E-Paper-Display. Der LDO des Entwicklerboards wandelt die 5 V auf die vom Mikrocontroller benötigten 3.3 V – eine zusätzliche Spannungsconversion, die unnötige Verlustleistung erzeugt und im Hardware Redesign beseitigt wird.

4.2.2 Hardware Redesign

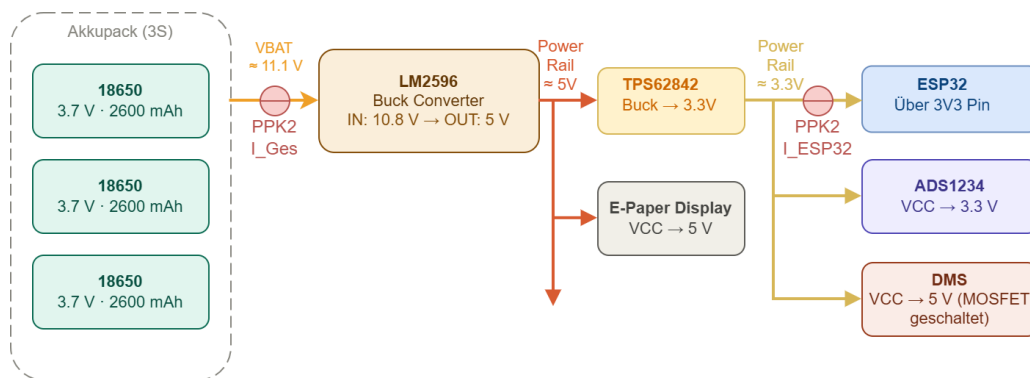


Abbildung 2: Schema Strompfad – Hardware Redesign

Das Hardware Redesign optimiert den Strompfad in zwei wesentlichen Punkten:

- **Dehnmessstreifen über MOSFET geschaltet:** In der Ausgangslage waren die Wägezellen dauerhaft mit Spannung versorgt – auch im Deep Sleep. Durch einen MOSFET wird die Versorgung der Dehnmessstreifen vor dem Eintreten in den Deep Sleep unterbrochen, was den Ruhestrom merklich senkt.

- **ESP32 direkt über den 3V3-Pin versorgt:** Das Entwicklerboard enthielt einen internen LDO, der 5 V auf 3.3 V regelte. Im neuen Design wird der ESP32 direkt über den 3V3-Pin eingespeist, womit diese Verlustleistung entfällt.

4.3 Strommessungen und Low-Power-Optimierung

Ein zentrales Ziel dieser Arbeit ist die Minimierung des Stromverbrauchs, um eine möglichst lange Batterielaufzeit zu erreichen. Die folgenden Messungen dokumentieren den Ladungsverbrauch in den einzelnen Betriebsphasen und quantifizieren den Effekt der implementierten Optimierungen. Tabellen 5 und 3 fassen die verbrauchte Ladung pro Phase zusammen – jeweils im direkten Vergleich zwischen Ausgangslage und optimiertem System.

4.4 Übersicht Zyklus

Die Strommessungen wurden mit dem *Nordic Semiconductor Power Profiler Kit II* (PPK2) in Verbindung mit der Software *nRF Connect for Desktop* (Power Profiler App) durchgeführt. Das PPK2 wurde als Amperemeter in den jeweiligen Versorgungspfad eingeschleift; das Gerät erfasst den Strom mit einer Abtastrate von bis zu 100,000 Samples/s, was eine präzise zeitliche Auflösung auch kurzer Stromspitzen ermöglicht. Da das PPK2 auf eine maximale Eingangsspannung von 5 V ausgelegt ist, kann es nicht direkt an der Batterie (11.1 V) betrieben werden; alle Messungen erfolgen daher am Ausgang des LM2596S-Schaltreglers [7].

Für diese Arbeit wurden zwei Messkonfigurationen eingesetzt:

- **Konfig A – ESP32 am 3,3 V-Pin:** Das PPK2 misst den Strom des ESP32 sowie der direkt angeschlossenen Messperipherie bei 3.3 V. Diese Konfiguration gilt für beide Systeme: Im neuen System ist der ESP32 ohnehin direkt am 3V3-Ausgang des LM2596S versorgt; beim alten System wurde ein separates Netzteil an den 3V3-Pin angeschlossen und die V_IN-Verbindung getrennt, um den internen LDO sowie den dauerhaft versorgten Dehnmessstreifen aus dem Messpfad auszuschliessen.
- **Konfig B – Altes System unverändert am 5 V-Rail:** Das PPK2 erfasst den Gesamtstrom des alten Systems am Ausgang des LM2596S (5 V) – einschliesslich LDO-Verlust (interne 5 V → 3,3 V-Konvertierung) und Dauerstrom der Dehnmessstreifen. Das neue System kann in dieser Konfiguration nicht gemessen werden, da es hardwareseitig direkt auf 3.3 V ausgelegt ist.

Vergleichsstrategie Zwei Vergleichstabellen werden eingesetzt:

- **Software-Vergleich** (Tabelle 5): Altes System unverdrahtet (Konfig A) gegen neues System (Konfig A), beide bei 3.3 V. Da das Spannungsniveau identisch ist, lassen sich mC-Werte direkt gegenüberstellen. LDO-Verlust und Dehnmessstreifen-Dauerstrom werden aus dem Messpfad ausgeschlossen, sodass ausschliesslich der Effekt der Software-Optimierungen sichtbar wird.
- **Gesamtsystem-Vergleich** (Tabelle 3): Altes System unverändert (Konfig B, 5 V) gegen neues System (Konfig A, 3.3 V). Die unterschiedlichen Spannungsniveaus machen einen direkten mC-Vergleich ungültig; der Verbrauch wird daher in mAh angegeben, und die vollständige Energiebilanz folgt in Abschnitt 4.8.

Im alten System sind LDO-Verlust und Dehnmessstreifen-Dauerstrom nicht vernachlässigbar und werden daher separat erfasst (Konfig B). Im neuen System entfällt der LDO, und die Dehnmessstreifen werden über ein MOSFET geschaltet – beide Verluste sind im Tiefschlaf vernachlässigbar, weshalb Konfig A für beide Vergleiche ausreicht.

In der nRF Connect Power Profiler App lassen sich Zeitfenster (*Selektionen*) manuell markieren, woraufhin die Software automatisch den mittleren Strom, die Dauer sowie den gesamten Ladungsverbrauch der Selektion in Millicoulomb (mC) und Milliampere-Stunden (mAh) berechnet. Alle in dieser Arbeit angegebenen Ladungswerte basieren auf solchen Selektionen aus aufgezeichneten Stromprofilen.

4.5 Software Optimierung

Abbildung 3 zeigt den Stromverlauf eines vollständigen Messzyklus in der Ausgangslage, wie er direkt aus der Vorgängerarbeit übernommen wurde. In dieser Vorarbeit stand die funktionale Korrektheit im Vordergrund – eine gezielte Optimierung des Stromverbrauchs hatte keine Priorität.

Im Profil sind drei charakteristische Abschnitte erkennbar: ein kurzer initialer Peak während Boot und Initialisierung, ein ruhigeres Plateau während der ADC-Messung sowie ein erhöhter und zeitlich ausgedehnter Abschnitt während der BLE-Kommunikation. Die gesamte Aktivphase erstreckt sich über ca. **10 s** mit einem Gesamtladungsverbrauch von **0.314 mAh**, bevor das System in den Deep Sleep übergeht.

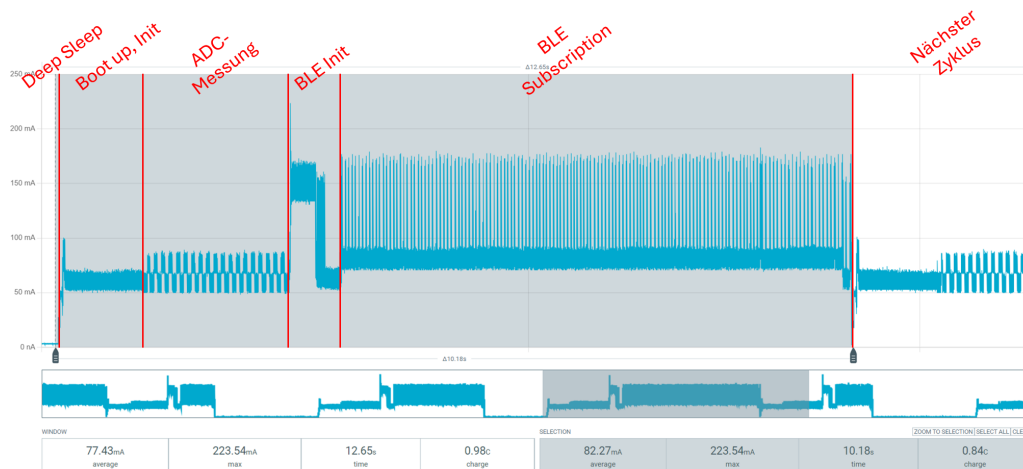


Abbildung 3: Stromverbrauch – Ausgangslage, ein vollständiger Messzyklus

1. **Deep Sleep:** Zwischen den Messzyklen befindet sich das System im Deep Sleep. Die meisten Funktionen sind abgeschaltet; lediglich der RTC-Timer läuft weiter und weckt den ESP32 nach Ablauf des konfigurierten Intervalls. Der Stromverbrauch ist in dieser Phase minimal.
2. **Boot und Initialisierung:** Der ESP32 startet nach dem Deep Sleep neu. Der nichtflüchtige Speicher (NVS) wird ausgelesen, die State Machine initialisiert und der HX711-ADC eingerichtet.
3. **ADC-Messung:** Der HX711 erfasst 20 Samples auf einem einzigen Kanal, an dem alle vier Wägezellen als gemeinsame Brücke angeschlossen sind. Zwischen jedem

Sample wartet der ESP32 in einem blockierenden `delay(50ms)`, was den Kern während der gesamten Messphase vollständig aktiv hält.

4. **BLE-Initialisierung:** Der BLE-Stack wird gestartet und das Advertisement für die Gateway-Verbindung gesendet.
5. **Verbindung und Datenübertragung:** Das System wartet auf den Verbindungsaufbau des Gateways (typisch ca. 6 s), überträgt den Messwert per GATT-Indication und trennt die Verbindung.
6. **Nächster Zyklus:** Der Messzyklus ist abgeschlossen; das System kehrt in den Deep Sleep zurück. Im gezeigten Stromprofil folgt der nächste Zyklus sofort, weshalb kein ausgeprägtes Tiefstromplateau sichtbar ist.

4.6 Gesamtsystem Stromverbrauch

Tabelle 5 vergleicht die Software-Ausgangslage mit dem optimierten System; beide wurden in Konfig A (3.3 V) gemessen, sodass die Ladungswerte in mC direkt gegenübergestellt werden können. Tabelle 3 stellt das alte Gesamtsystem (Konfig B, 5 V) dem neuen System (Konfig A, 3.3 V) gegenüber; da die Spannungsniveaus differieren, wird der Verbrauch in mAh angegeben.

4.6.1 Vergleich Energieverbrauch Aktiver Zyklus

Tabelle 3: Gesamtsystem-Vergleich (Aktiver Zyklus)			
Betriebsphase	Alt [mJ]	Neu [mJ]	Ersparnis
Vor Buck			
Verlust Buck (Berechnet)	2955		
ESP32 über LDO	3430	nA	
ESP32 über 3V3	2220	223.8	
davon ESP32 LDO		nA	
Dennmessstreifen		89.1	
ADC			
Aktivphase gesamt	—	0.314	0.039
Gesamt (ein Zyklus)			

4.6.2 Vergleich Leistungsverbrauch Deep-Sleep

Tabelle 4: 5V und 3V Rail hinter Buck converter - Vergleich (Deep-Sleep)

Betriebsphase	Alt [mW]	Neu [mW]	Ersparnis
Nach Buck-Conv. Dehnmessstreifen	78.19	0	
ADC	9.762	4.042	
ESP32	27.53	10.15	
Nach Buck summiert	115.4	14.19	
Gesamt (Vor Buck)	268.2	90.96	

Der Buck converter hat bereits im Leerlauf einen Verbrauch von 69.77 mW. Der Stromverbrauch ist verglichen mit den 14.19 mW des neuen Systems gigantisch.

4.6.3 Vergleich Energieverbrauch ESP32

Tabelle 5: Software-Vergleich: Altes System umverdrahtet vs. Neues System (beide Konfig A, 3.3 V) am 3V3 des ESP32. Beim alten System sind eigentlich 5V vom Buck convter am VIN Pin des Esp32 angeschlossen, der die Spannung auf 3.3Volt regelt, für den direkten Vergleich werden aber beide 3.0V am 3v3 angeschlossen, wie es beim neuen optimierten Waage festverdrahtet ist. Die Denmmessstreifen werden ebenfalls abehängt biem neuen System, biem alten System sind die DMS am HX711 Modul festverdrahtet, welcher sich nicht in der Messschleufe befindet, der Verlust vom ADS1234 ADC beim neuen System wird dank eines Messpins noch seperat betrachtet.

Betriebsphase	Alt [mJ]	Neu [mJ]	Einsparung
Boot und Ramp-up	360.3	71.08	−80 %
ADC-Messung	807,8	50.83	−83 %
BLE	1293	101.6	−93 %
ADC Chip	–	13.42	–
Gesamt(ein Zyklus)	689,4	76,3	−89 %

Die gesamte Aktivphase beträgt in der Ausgangslage ca. **10 Sekunden** pro Zyklus mit einem Ladungsverbrauch von **0.314 mAh** (Abbildung 3).

Die nachfolgenden Abschnitte beschreiben jede Betriebsphase im Detail – mit Stromprofilen der Ausgangslage sowie der optimierten Version – und erläutern, welche Massnahmen zur jeweiligen Einsparung geführt haben:

1. Ramp Up – Boot, Initialisierung und ADC (Abschnitt ??):

Diese Phase umfasst den ESP32-Bootvorgang, die State-Machine-Initialisierung sowie die ADS1234-Messung. Die Ausgangslage verbrauchte **0.123 mAh**. Optimierungspotenzial liegt unter anderem in der Reduktion der Boot-Zeit sowie in der Minimierung der ADC-Einschaltdauer.

2. ADC (Abschnitt ??):

In der Ausgangslage verwendete die Vorgängerarbeit den **HX711** – einen einkanali- gen 24-Bit-ADC, an den alle vier Wägezellen als gemeinsame Brücke angeschlossen waren. Die Messung erfasste 20 Samples auf diesem einzigen Kanal; zwischen jedem Sample wurde ein blockierendes **delay**(50 ms) ausgeführt, was den ESP32-Kern für die gesamte Wartezeit vollaktiv liess.

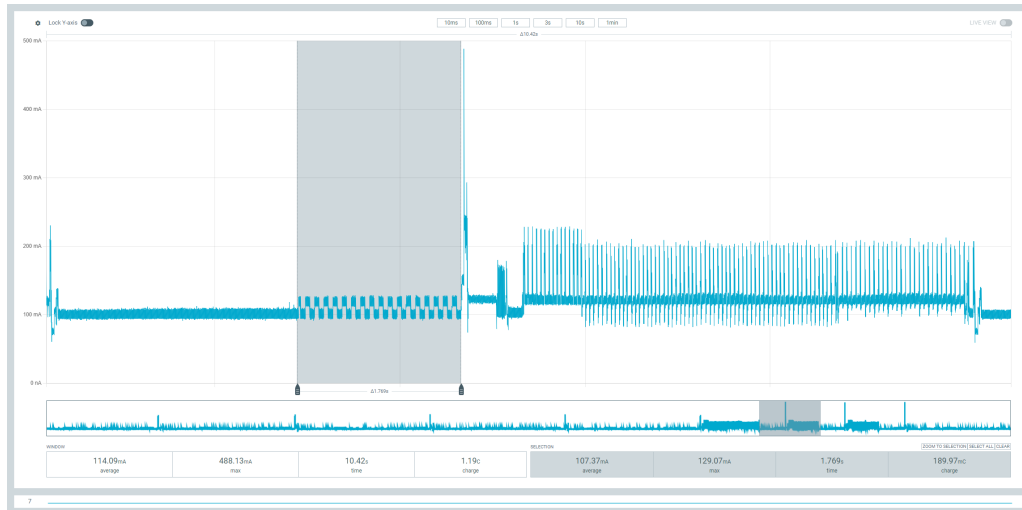


Abbildung 4: Stromverlauf – ADC (Ausgangslage)

Im optimierten Design werden die vier Kanäle des ADS1234 sequenziell ausgelesen; pro Kanal werden **5 Samples** erfasst. Nach jedem abgeschlossenen Sample kehrt der ESP32 sofort in den Light Sleep zurück und wird durch einen GPIO-Interrupt auf dem **DRDY**-Pin geweckt, sobald der ADS1234 die nächste Wandlung bereitstellt. Im Profil sind diese fünf kurzen Aktivierungsspitzen pro Kanalgruppe deutlich erkennbar.

Zwischen den vier Kanalgruppen zeigt sich jeweils eine Pause von ca. **400 ms**: Dies ist die physikalisch bedingte Einschwingzeit des internen Sigma-Delta-Filters nach einem Kanalwechsel. Der ADS1234 integriert einen digitalen sinc³-Filter; dessen Impulsantwort erstreckt sich über 3–4 Ausgangsperioden, was bei der eingestellten Datenrate von 10 Hz rund 300–400 ms entspricht. Erst nach dieser Zeit sind die Ausgabewerte frei von Einflüssen des vorangegangenen Kanals. Auch diese Wartezeit verbringt der ESP32 im Light Sleep – der Kern schläft, bis **DRDY** das Ende der Einschwingphase signalisiert.

Der Gesamtladungsverbrauch der ADC-Phase sinkt von ca. **190 mC** auf ca. **100 mC** – eine Einsparung von rund **47 %**.

3. BLE-Initialisierung und Advertisement (Abschnitt ??):

In der Ausgangslage verwendete die Firmware die Arduino-BLE-Bibliothek (Blue- droid) und baute für jeden Messzyklus eine vollständige GATT-Verbindung mit dem Gateway auf: Der ESP32 initialisierte den BLE-Stack, startete das Advertisement und wartete, bis das Gateway sich verbindet und den Messwert per Indication abon- niert. Die Dauer dieser Phase variiert stark in Abhängigkeit vom Scan-Intervall des Gateways – im gezeigten Profil **5.6 s** bei **0.7 C**, in anderen Messungen bis zu **7 s** bei **0.83 C**.

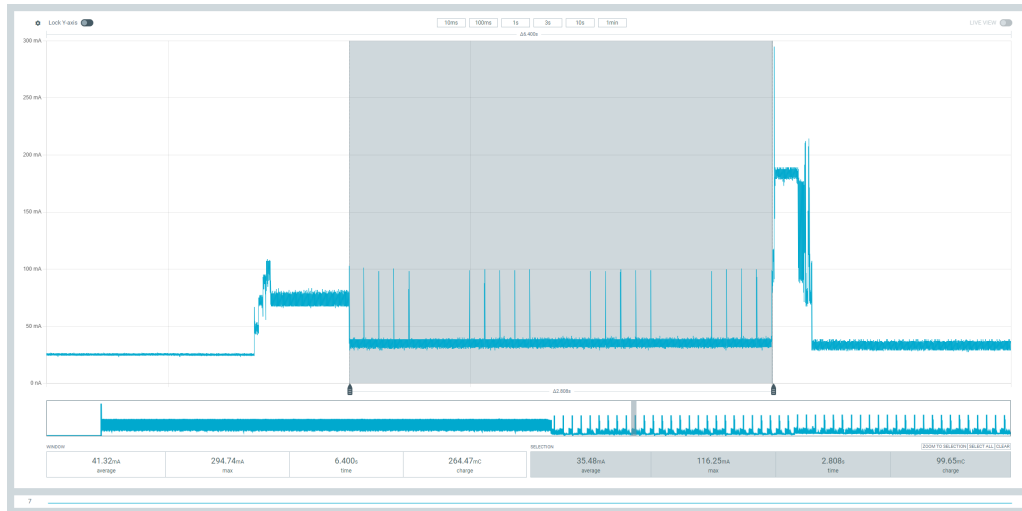


Abbildung 5: Stromverlauf – ADC (Optimiert)

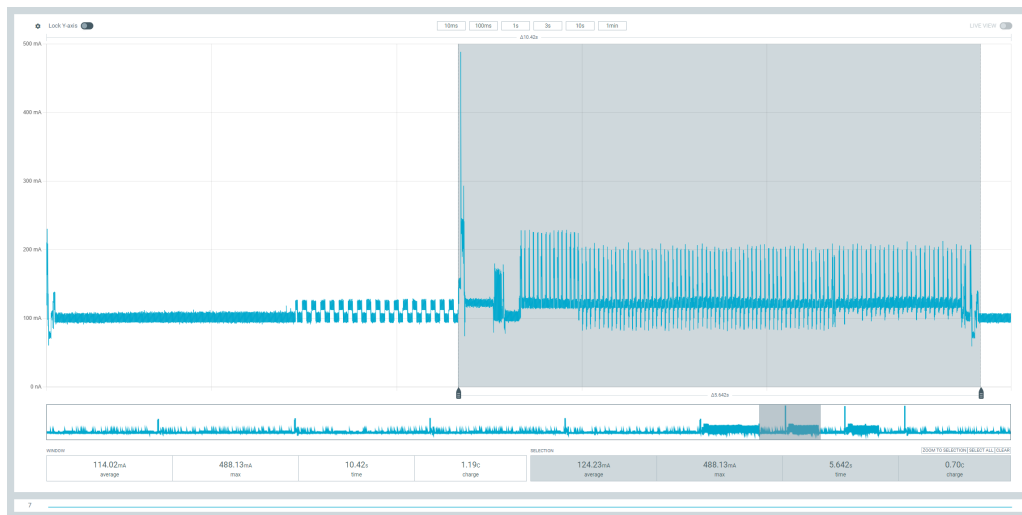


Abbildung 6: Stromverlauf – BLE Initialisierung und Subscription (Ausgangslage - Arduino BLE-Library)

Der fundamentale Ansatz der Optimierung bestand darin, den Verbindungsaufbau vollständig zu eliminieren: Der Messwert wird direkt in die Manufacturer-Specific-Data des Advertisement-Pakets kodiert – das Gateway empfängt die Daten durch passives Scannen, ohne eine Verbindung aufzubauen. Als erster Schritt wurde dieser Ansatz noch mit der bestehenden Arduino-BLE-Bibliothek umgesetzt; Init und zwei Advertisements benötigten **728 ms** und **78.8 mC**.

Im zweiten Schritt wurde die schwergewichtige Bluedroid-Implementierung durch den schlanken **NimBLE**-Stack ersetzt, der speziell für ressourcenbeschränkte Geräte entwickelt wurde und einen deutlich geringeren Initialisierungs-overhead aufweist. Damit sinkt der Ladungsverbrauch für Init und zwei Advertisements auf ca. **40 mC**.

Die Wahl von **zwei Advertisements** pro Zyklus ist ein bewusster Kompromiss: Da die Sendezeit in NimBLE nur als Zeitwert konfigurierbar ist und nicht als diskrete Paketanzahl, werden innerhalb des gewählten Zeitfensters typischerweise zwei Pakete ausgesendet. In einem Zuverlässigkeitstest über **100 aufeinanderfolgende**

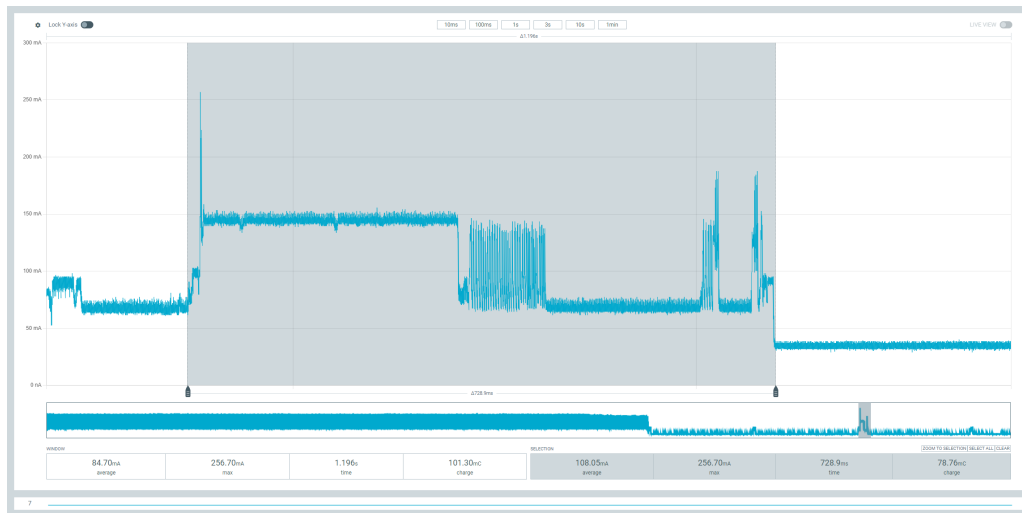


Abbildung 7: Stromverlauf – BLE Initialisierung und Advertisement (Optimiert - Arduino BLE-Library)

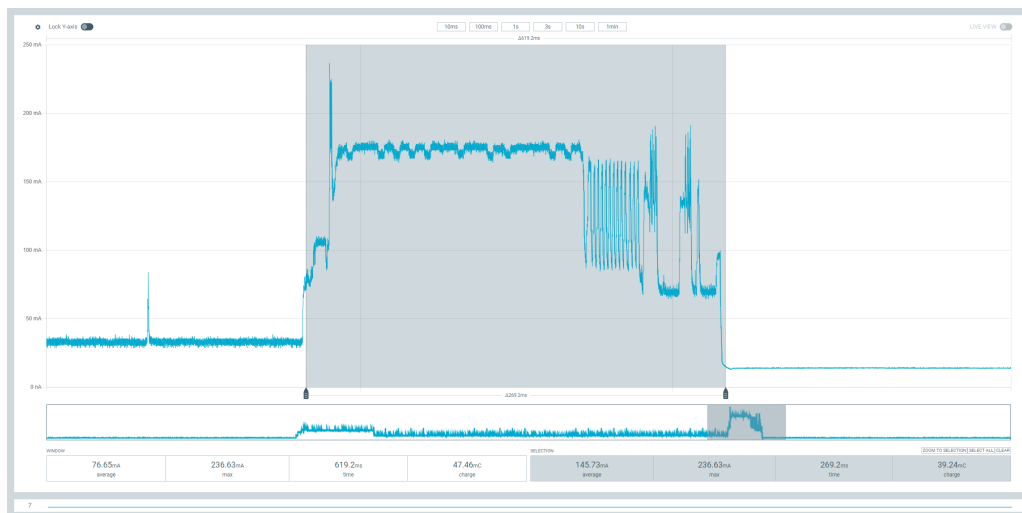


Abbildung 8: Stromverlauf – BLE Initialisierung und Advertisement (Optimiert - NimBLE)

Messzyklen wurden **91 von 100** Advertisements erfolgreich vom Gateway empfangen, was für ein periodisches Überwachungssystem ausreichend ist.

4. BLE-Verbindung und Datenübertragung (Abschnitt ??):

Das Gateway baut die Verbindung auf, der Messwert wird per Indication übertragen und die Verbindung anschliessend getrennt. In der Ausgangslage betrug der Ladungsverbrauch dieser Phase (gemeinsam mit BLE-Init gemessen) **0.194 mAh**. Durch Optimierung der Connection-Parameter und Reduktion der Wartezeit auf den Gateway konnte dieser Wert gesenkt werden.

4.7 Verluste Hardware

Dieses Kapitel bezieht sich auf die Verluste vom ADC, Buck converter und LDO

```

info: EdgeDevice.BLE.GWBackgroundService[0]
      Advertisement #324 received. Window: 91/100 received (9 missed). Total: 265 received, 27 missed.
info: EdgeDevice.BLE.GWBackgroundService[0]
      Advertisement weight received: -114976.74 g from 14:2B:2F:EC:1A:EA

```

Abbildung 9: Zuverlässigkeitstest – 100 BLE Advertisements (Optimiert - NimBLE)

4.7.1 Methodik

Es wird jeweils über einen Zyklus die Energie vor und nach dem LDO und Buck converter gemessen, die Differenz dieser Energien ist dann dem Bauteil zu verschulden. Bei den ADC wird dann einfach der Strompfad zum Bauteil angeschaut.

4.7.2 Buck converter

Zur Bestimmung des Verlusts am LM2596S-Schaltregler wird die Energie *vor* dem Konverter (Eingang, Batterieseite) mit der Energie *nach* dem Konverter (Ausgang, Lastseite) verglichen; die Differenz entspricht dem dem Schaltregler-Board zuzuschreibenden Verlust. Die Werte stammen jeweils aus der Tabelle des Kapitels *Übersicht Stromverbrauch* und umfassen einen vollständigen Messzyklus. Beim alten System wurde sichergestellt, dass BLE-Initialisierung und Subscription zusammen ca. 5.4 s dauerten, um vergleichbare und reproduzierbare Messungen zu gewährleisten.

Altes System

$$\Delta E_{\text{Buck, alt}} = 6503 \text{ mJ} - 3548 \text{ mJ} = 2955 \text{ mJ} \quad (6)$$

$$\eta_{\text{Buck, alt}} = \frac{3548 \text{ mJ}}{6503 \text{ mJ}} \approx 54.6 \% \quad (7)$$

Neues System

$$\Delta E_{\text{Buck, neu}} = 689.5 \text{ mJ} - 343.4 \text{ mJ} = 346.1 \text{ mJ} \quad (8)$$

$$\eta_{\text{Buck, neu}} = \frac{343.4 \text{ mJ}}{689.5 \text{ mJ}} \approx 49.8 \% \quad (9)$$

Vergleich Beide Systeme weisen einen ähnlichen Wirkungsgrad von ca. 50 % auf. Der entscheidende Unterschied liegt im absoluten Verlust: Das alte System verliert 2955 mJ pro Zyklus am Schaltregler, das neue System nur 346.1 mJ – eine Reduktion um den Faktor **8,5**. Der geringere absolute Verlust ist direkt auf den deutlich niedrigeren Gesamtenergieverbrauch des optimierten Systems zurückzuführen.

Plausibilitätsprüfung anhand des Datenblatts Das Datenblatt des LM2596S [7] gibt für eine Konfiguration von 12 V Eingang auf 3.3 V Ausgang bei einem Ausgangsstrom von 3 A einen typischen Wirkungsgrad von 79 % an. Die gemessenen Werte von ca. 50 % liegen erwartungsgemäss darunter – zum einen weil beide Systeme mit mittleren Lastströmen weit unter 3 A betrieben werden, zum anderen weil es sich nicht um den nackten LM2596S-Chip handelt, sondern um ein fertiges Schaltregler-Board mit zusätzlicher Peripherie (Freilaufdiode, Spule, Filterkapazitäten, Schutzschaltungen), die den Gesamtwirkungsgrad weiter senken.

Bei geringer Nutzlast dominiert zudem der Quiescent-Strom des LM2596S von ca. 5.5 mA den Eingangsstrom überproportional. Für das neue System mit ca. 35.4 mA mittlerem Laststrom entspricht der Quiescent-Strom bereits rund 13 % des Gesamteingangsstroms – bei 3 A wäre er vernachlässigbar. Da der LM2596S keinen Burst- oder PFM-Modus für Leichtlast unterstützt, ist dieser Verlust strukturell bedingt. Die Messung bestätigt qualitativ die in Abschnitt 4.8 verwendete Effizienzannahme von 61 % [7].

4.7.3 ADC

Der ADC wird vom Programm per !PWDN Pin in den Schlaf geschickt nach der Messung und davor geweckt, aber dadurch dass der ESP32 in den Deep Sleep geht, werden die GPIO floating gelassen. Es hätte ein Pull down Widerstand eingebaut werden müssen, dass er auch während des Sleeps abgeschaltet wird, dadurch konsumiert er im Deep Sleep 1mA. Während des Zyklus verbraucht er 13.42 mJ. Etwas mehr als nötig, da er bereits eingeschaltet aus dem Deep Sleep kommt und nur kurz für die BLE Phase abgeschaltet wird.

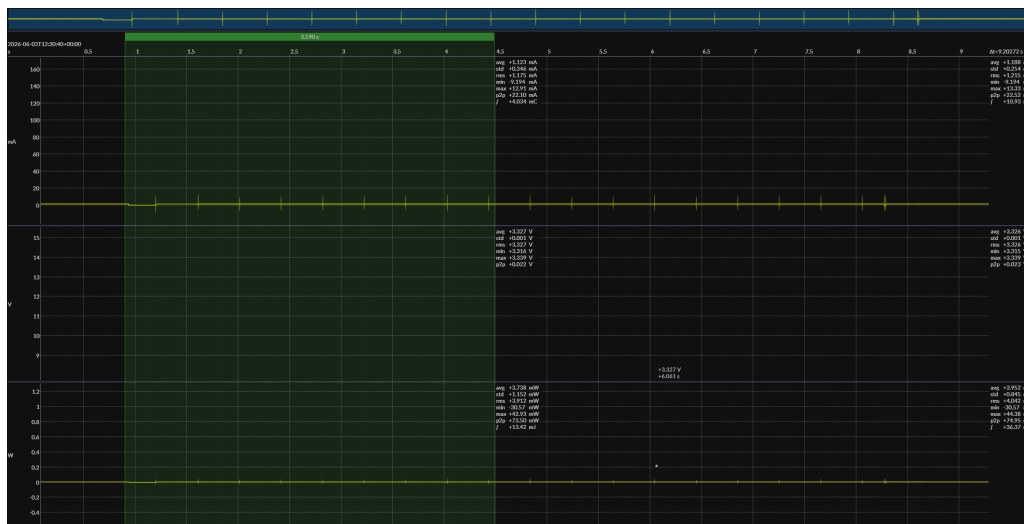


Abbildung 10: Stromverlauf – ADC Chip(Neues Design))

4.8 Gesamtverbrauch und Batterielebensdauer

Die Batterielaufzeit ergibt sich aus der Energiebilanz pro Messzyklus, skaliert über die Effizienz des LM2596S-Schaltreglers [7] auf Batterieebene. Als Batteriekapazität wird ein typischer Wert von $3 \times 2600 \text{ mAh}$ bei 11.1 V angenommen:

$$E_{\text{Batterie}} = 2600 \text{ mA h} \times 11.1 \text{ V} \approx 28.86 \text{ W h} \approx 103.9 \text{ kJ} \quad (10)$$

Aktivphase Die Aktivphase des optimierten Systems umfasst ADC-Messung (**100 mC**) und BLE-Advertisement (**40 mC**), gemessen bei 3.3 V:

$$E_{\text{aktiv}} = (Q_{\text{ADC}} + Q_{\text{BLE}}) \cdot 3.3 \text{ V} = 140 \text{ mC} \cdot 3.3 \text{ V} \approx 462 \text{ mJ} \quad (11)$$

Tiefschlaf Der gemessene Ruhestrom am 3.3 V-Rail beträgt **13,75 mA**:

$$P_{\text{sleep}} = 13.75 \text{ mA} \cdot 3.3 \text{ V} \approx 45.4 \text{ mW} \quad (12)$$

Energiebilanz pro Zyklus Bei einem Messintervall T und einer Aktivphase von ca. $t_{\text{aktiv}} \approx 5$ s ergibt sich am 3,3 V-Rail:

$$E_{\text{Zyklus, Rail}} = E_{\text{aktiv}} + P_{\text{sleep}} \cdot (T - t_{\text{aktiv}}) \quad (13)$$

Der LM2596S erreicht bei der durchschnittlichen Last des neuen Systems (3.3 V, 35.4 mA) eine Effizienz von ca. **61 %** [7]. Der tiefe Wirkungsgrad ist auf den Quiescent-Strom des LM2596S von ca. 5.5 mA zurückzuführen, der bei geringer Nutzlast – insbesondere im Tiefschlaf – überproportional ins Gewicht fällt. Die Skalierung auf Batteriebene und die resultierende Laufzeit ergeben sich zu:

$$E_{\text{Zyklus, Bat}} = \frac{E_{\text{Zyklus, Rail}}}{\eta_{\text{Buck}}} \Rightarrow t_{\text{Laufzeit}} = \frac{E_{\text{Batterie}}}{E_{\text{Zyklus, Bat}}} \cdot T \quad (14)$$

Tabelle ?? fasst die Laufzeiten für verschiedene Messintervalle zusammen.

Tabelle 6: Berechnete Batterielaufzeit – Optimiertes System ($\eta_{\text{Buck}} = 61\%$, 3×2600 mAh bei 11.1 V)

Intervall T	$E_{\text{Zyklus, Rail}}$ [J]	$E_{\text{Zyklus, Bat}}$ [J]	Laufzeit
1 min (60 s)	2,96	4,85	≈ 15 Tage
10 min (600 s)	27,5	45,0	≈ 16 Tage
30 min (1800 s)	81,9	134,3	≈ 16 Tage

Die berechneten Laufzeiten sind nahezu unabhängig vom Messintervall: Der Tiefschlaf-Ruhestrom von 13.75 mA dominiert die Energiebilanz ab einem Intervall von wenigen Minuten. Das grösste verbleibende Einsparpotenzial liegt daher in der Reduktion des Tiefschlaf-Ruhestroms, beispielsweise durch einen modernen Schaltregler mit Leichtlast-Optimierung wie den TPS62842 [5].

Vergleich mit der Ausgangslage Auf Batteriebene ergibt sich ein direkter Vergleich aus den gemessenen Durchschnittsströmen. Das alte System wies am 5 V-Rail einen mittleren Strom von **104 mA** auf; mit einer LM2596S-Effizienz von **79 %** entspricht das einem Batteriestrom von ca. **59 mA**. Das optimierte System erreicht am 3,3 V-Rail **35,4 mA**; mit $\eta = 61\%$ entspricht das ca. **17 mA** auf Batteriebene – eine Reduktion um den Faktor **3,5**. Die Batterielaufzeit steigt damit von ca. **44 h** ($\approx 1,8$ Tage) auf ca. **150 h** ($\approx 6,3$ Tage).

5 Messungen

5.1 Messgenauigkeit

5.2 Low-Power Analyse und Optimierung

5.2.1 Vergleich Energieverbrauch Aktiver Zyklus

Alt: ESP32 über LDO:

ESP32 über 3v3 Pin:

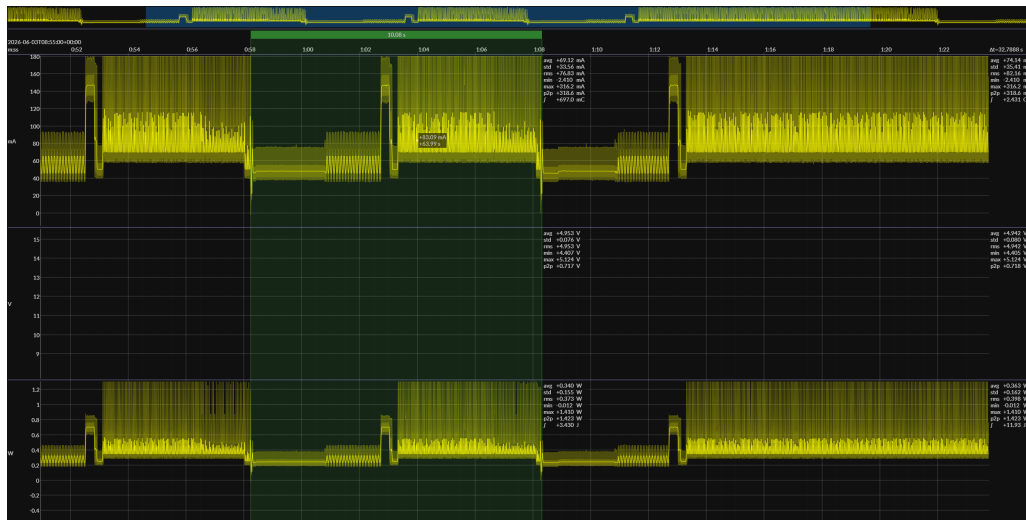


Abbildung 11: Messpunkt: vor ESP32 VIN Pin (LDO) (Alt))

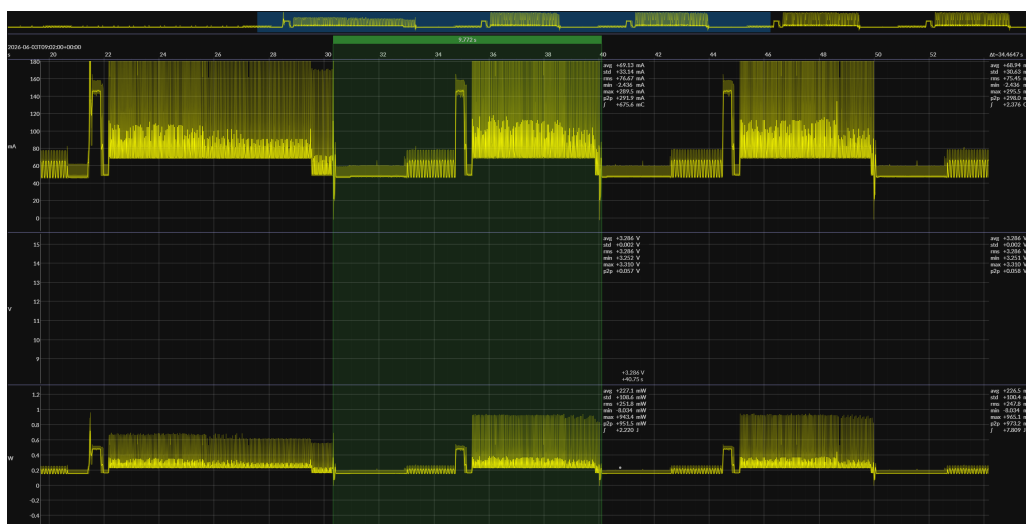


Abbildung 12: Messpunkt: vor ESP32 VIN Pin (LDO) (Alt))

5.2.2 Vergleich Leistungsverbrauch Deep-Sleep

5.2.3 Vergleich Energieverbrauch ESP32

6 Fazit, Diskussion und Ausblick

Literatur

- [1] esp32-c3-wroom-02 datasheet. https://documentation.espressif.com/esp32-c3-wroom-02_datasheet_en.pdf/. Online; aufgerufen 01.02.2026.
- [2] esp32-s3-wroom-02 datasheet. https://documentation.espressif.com/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf/. Online; aufgerufen 01.02.2026.
- [3] Getting started with esp32-c3 super mini. <https://randomnerdtutorials.com/getting-started-esp32-c3-super-mini/>. Online; aufgerufen 01.02.2026.

- [4] Kraftsensor 0...1 kg, yzc-133. https://www.reichelt.com/ch/de/shop/produkt/kraftsensor_0_1_kg_ymc-133-282443. Online; aufgerufen 29.04.2026.
- [5] Tps62842 datasheet. <https://www.ti.com/product/TPS62840/part-details/TPS62842DGRR>. Online; aufgerufen 01.06.2026.
- [6] Texas Instruments. ADS1234 product datasheet. <https://www.ti.com/lit/ds/symlink/ads1234.pdf>. Online; aufgerufen 29.04.2026.
- [7] Texas Instruments. LM2596 SIMPLE SWITCHER® Power Converter 150-kHz 3-A Step-Down Voltage Regulator. <https://www.ti.com/lit/ds/symlink/lm2596.pdf>. Revision C (SNVS124C); Online; aufgerufen 01.06.2026.